

 UNIVERSIDAD DON BOSCO	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERIA ESCUELA DE COMPUTACIÓN
CICLO 2	Materia: Programación Orientada a Objetos (POO404) Guia de Laboratorio No.4 Tema: Programacion orientada a objetos (POO) Parte 2

I. OBJETIVOS.

Que el estudiante:

- Implemente esquemas de Clases basadas en Herencia.
- Diseñe aplicaciones Java con esquemas de herencia de clases
- Diseñe clases estáticas, las cuales se deben llamar directamente, sin ser instanciadas.
- Implemente el principio de Polimorfismo.

II. INTRODUCCIÓN.

Herencia

La **Herencia** es uno de los 4 pilares de la programación orientada a objetos (POO) junto con la **Abstracción**, **Encapsulación** y **Polimorfismo**.

La herencia es una forma de reutilización de software en la que las clases se crean absorbiendo los miembros (atributos y métodos) de una clase ya existente, y se mejoran con las nuevas capacidades, o con modificaciones en las capacidades ya existentes.

Al crear una clase basada en otra, en vez de declarar miembros (variables y métodos) completamente nuevos, el programador puede designar que la nueva clase herede los miembros de una clase existente.

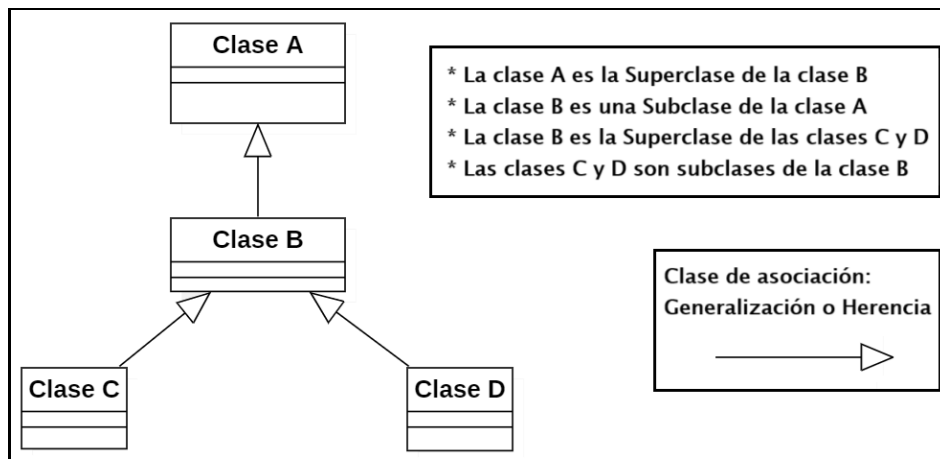
Clase Base y Clases Derivadas

A la clase inicial utilizada para crear a otras clases se le conoce como **Clase Base o Superclase o Clase Padre**, y a cada nueva clase creada a partir de una Clase Base se les conoce como **Clase Derivada o Subclase o Clase Hija**. Una vez creada una subclase, esta a su vez puede convertirse en superclase de futuras subclases.

Una subclase generalmente agrega sus propias variables y métodos. Por lo tanto, una subclase es más específica que su superclase y representa a un grupo más especializado de objetos.

Generalmente, la subclase exhibe los comportamientos de su superclase junto con comportamientos adicionales específicos de esta subclase.

Un esquema de herencia de clases se puede representar en UML, tal como muestra el siguiente ejemplo:



Ejemplo de Esquema UML de Clases

Tal como muestra el diagrama UML, las relaciones de herencia forman estructuras jerárquicas en forma de árbol. Una superclase existe en una relación jerárquica con sus subclases. Aunque las clases pueden existir de manera independiente, cuando participan en relaciones de herencia se afilian con otras clases.

Una clase se convierte ya sea en una superclase proporcionando datos y comportamientos a otras clases, o en una subclase, heredando sus datos y comportamientos de otras clases.

En UML, la relación de *generalización* o también conocida como *herencia* es un tipo de relación entre clasificadores (como clases) donde una clase (subclase) se basa en otra (la superclase). La subclase hereda atributos, operaciones y relaciones de la superclase.

La generalización se utiliza para modelar relaciones "**es un tipo de**". La subclase es un tipo especializado de la superclase.

La generalización se representa con una línea sólida que termina en una flecha hueca apuntando a la superclase.

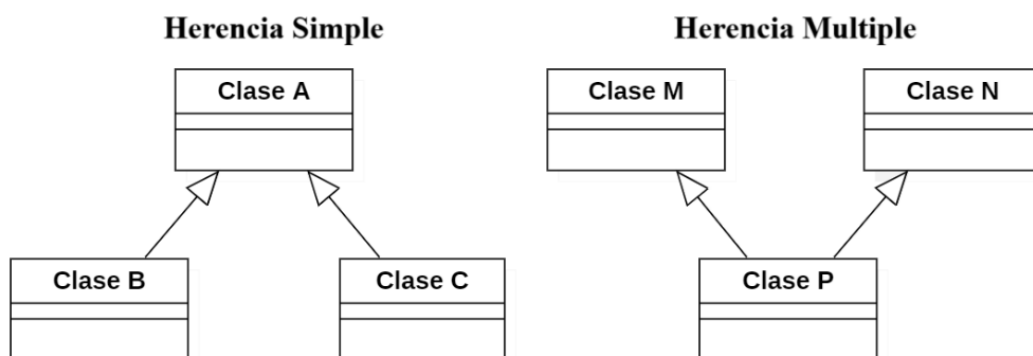
Tipos de Herencia

- **Herencia simple :**

Una clase posee una sola superclase directa. El gráfico de herencia es un árbol

- **Herencia múltiple:**

Una clase posee varias superclases directas. El gráfico de herencia no es un árbol



Herencia en Java

En java, la jerarquía de clases empieza con la clase **Object** (en el paquete **java.lang**), a partir de la cual heredan todas las clases en Java, ya sea en forma directa o indirecta. En el caso de la herencia simple, una clase se deriva de una superclase Java.

Java, a diferencia de C++, no soporta la herencia múltiple, una clase solo puede derivar de una única clase. Sin embargo, es posible "simular" la herencia múltiple con base en las **interfaces**, que se verán mas adelante.

Implementando Herencia (extends)

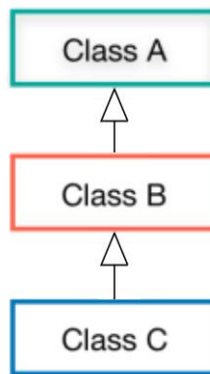
La palabra reservada **extends** permite implementar el concepto de herencia en Java. Se incluye para que una clase herede de otra clase.

Por ejemplo, en el caso de jerarquía de herencia mostrada en la imagen a continuacion, primero se debe definir a la clase A.

Luego, para definir a la clase B como derivada de clase A, se hace uso de la palabra **extends**, para que B herede miembros de la clase A.

Y por ultimo, al definir a la clase C, se utiliza nuevamente **extends** para definirla como subclase de la clase B.

Implementacion de Herencia en Java

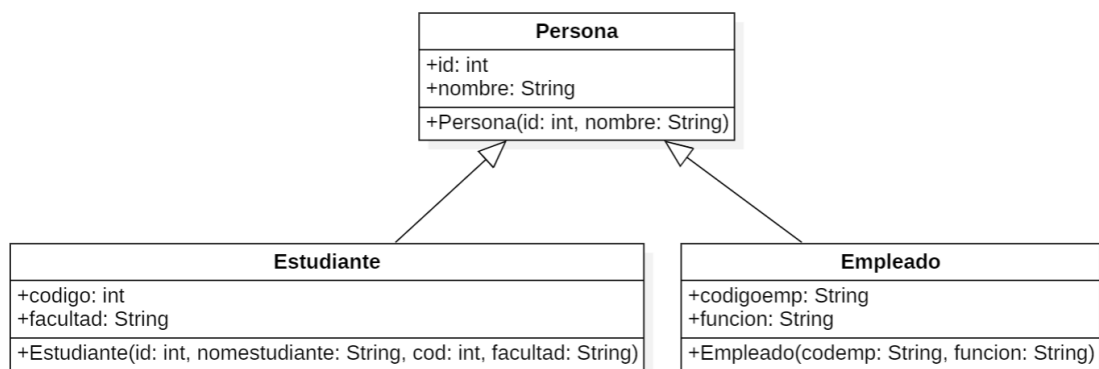


```
public class A {  
    .....  
}  
  
public class B extends A {  
    .....  
}  
  
public class C extends B {  
    .....  
}
```

Acceso a miembros heredados (super)

La palabra reservada **super** es utilizada en los miembros de la clase Derivada para acceder a los miembros implementados en la Superclase. Esta sentencia es comúnmente utilizada para acceder al constructor de la clase superior desde el constructor de la clase inferior.

Un ejemplo del concepto de herencia puede ser considerando, los miembros de una institucion de educacion. La institución está conformada por personas, pero cada persona tiene un rol dentro de la institución, que podría ser de empleado o estudiante, como se indica en el siguiente UML:



Para el ejemplo anterior, considere a un Estudiante, el cual "es una" Persona. Un ejemplo de implementación en Java de la herencia Persona-Estudiante se muestra a continuación. Observe el uso de las palabras **extends** para definir herencia y de **super** en la subclase, para invocar al constructor de la superclase:

```
public class Persona { //superclase
    public int id;
    public String nombre;

    public Persona(int id, String nombre){ //metodo constructor de superclase
        this.id = id;
        this.nombre=nombre;
    }
}

//subclase o clase derivada de superclase Persona
public class Estudiante extends Persona {
    //campos locales de la subclase Persona
    public int codigo;
    public String facultad;
    //constructor de subclase
    public Estudiante(
        int id, String nombreestudiante, int cod, String facultad){
        //usa palabra "super" para invocar a constructor de superclase Persona
        //envia sus argumentos para argumentos del constructor de superclase
        super(id,nombreestudiante);
        //asigna campos locales
        this.codigo = cod;
        this.facultad = facultad;
    }
}
```

Encapsulamiento con Herencia

Tal como se ha visto en la introducción de POO en Java, los 4 modificadores de acceso (**public**, **protected**, **private**) son palabras reservadas que se utilizan para controlar la *visibilidad o accesibilidad* de clases, métodos, variables, constructores y otros miembros de una clase.

De los 4 *modificadores de acceso*, ahora se describirá el uso de **protected**, utilizado solamente bajo la herencia de clases.

Modificador	Clase	Package	Subclase	Otros
public	✓	✓	✓	✓
protected	✓	✓	✓	•
default	✓	✓	•	•
private	✓	•	•	•

Restricción de modificadores de acceso de Java

Modificador	Descripcion
public	Permite que los elementos sean accesibles desde cualquier parte del código, tanto dentro como fuera de la clase.
private	Permite que los elementos sean accesibles solo desde dentro de la clase en la que se definen
protected	Permite que los elementos sean accesibles desde la clase en la que se definen y <u>desde cualquier clase que herede de ella</u>
default	Permite que los elementos sean accesibles solo dentro del paquete en el que se encuentran

Acceso entre paquetes

En Java, la palabra clave **protected** se utiliza como modificador de acceso para miembros de una clase (campos, métodos, constructores). Permite que esos miembros sean accesibles dentro del mismo paquete y también por subclases, incluso si estas están en diferentes paquetes. Esto significa que la subclase puede utilizar esos miembros como si fueran propios, incluso si no están en el mismo paquete que la superclase.

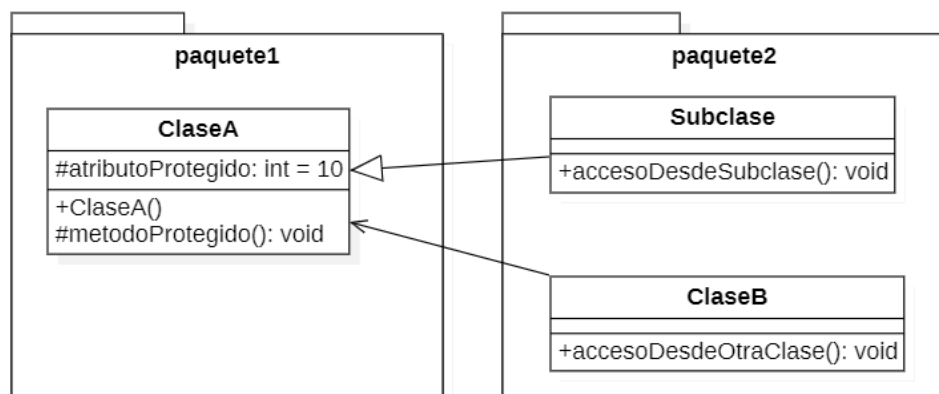
Es un nivel de acceso intermedio entre **private** (solo accesible dentro de la clase) y **public** (accesible desde cualquier lugar). Por lo tanto, **protected** permite el acceso a miembros desde:

- * La propia clase.
- * Clases dentro del mismo paquete.
- * Subclases (incluso en diferentes paquetes)

Restricciones:

Los miembros **protected** no son accesibles por clases que no son subclases y que no están en el mismo paquete. Y no se accede a miembros definidos como **protected** desde objetos de la clase base ni de objetos de las clases heredadas.

Ejemplo de miembros protegidos:



En UML, los miembros protegidos son marcados con el símbolo (#). Observe que las clases **Subclase** y **ClaseB** están en un paquete distinto al paquete donde está definida la clase **ClaseA**.

La clase Subclase es derivada de la superclase ClaseA.

En cambio, la clase ClaseB solo esta relacionada a la ClaseA, porque en alguno de sus miembros, define un objeto de la ClaseA, no hay generalización/herencia.

<pre>package paquete1; public class ClaseA { protected int atributoProtegido; public ClaseA(){ atributoProtegido = 10; } protected void metodoProtegido(){ System.out.println("Metodo protegido de Clase A"); } }</pre>	<pre>package paquete2; import paquete1.ClaseA; public class Subclase extends ClaseA { public void accesoDesdeSubclase(){ //acceso a miembros protegidos de ClaseA atributoProtegido = 25; metodoProtegido(); } }</pre>
<pre>package paquete2; import paquete1.ClaseA; public class ClaseB { //sin herencia a clase ClaseA public void accesoDesdeOtraClase(){ ClaseA objetoA = new ClaseA(); //Error: no se puede acceder directamente desde // otra clase sin herencia a ClaseA objetoA.atributoProtegido = 43; } }</pre>	

Clases Estaticas

Una **clase estática** es aquella que no puede ser instanciada, es decir, no se pueden crear objetos de dicha clase.

Todos los miembros (métodos y variables) de una clase estática deben ser también estáticos. Las clases estáticas se utilizan para agrupar funcionalidades que no dependen de instancias de objetos, como métodos de utilidad general.

Características principales de las clases estáticas

- **No instanciables:** No se pueden crear objetos de una clase estática usando el operador new.
- **Miembros estáticos:** Todos los miembros (métodos, variables, etc.) de una clase estática deben ser declarados como static.
- **Acceso a través de la clase:** Los miembros de una clase estática se acceden directamente a través del nombre de la clase, no a través de una instancia.
- **Utilidad:** Se utilizan para agrupar funcionalidades que no están asociadas a objetos específicos, como funciones de utilidad.
- **No pueden heredar de otras clases:** en el caso particular de Java, solo esta permitido heredar de la clase genérica Object:

- Las clases estáticas no pueden ser extendidas por otras clases ni implementar interfaces.
- **Pueden tener un constructor estático:** Aunque no se pueden instanciar, pueden tener un constructor estático para inicializar sus miembros estáticos.

En UML, a un metodo estático se le representa con un subrayado sobre todo su prototipo.

Uno de los posibles usos del modificador **static** es compartir el valor de un *campo* entre objetos de una misma clase. Si declara un campo estatico dentro de una clase, todos los objetos que declaremos basándonos en esa clase compartirán el valor de ese campo al que se le haya aplicado el modificador static, y se podrá modificar el valor de este desde todas las instancias de esta clase.

Ejemplo de aplicación de clases estáticas en Java

En Java, se puede definir como estático a un miembro de una clase o a la clase misma. Para ello, se utiliza la palabra reservada **static**. Por ej:

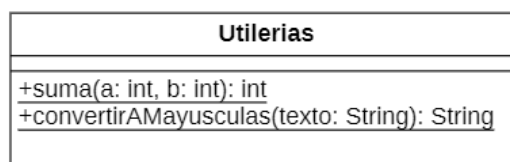
```
public class Utilerias {
    public static int suma(int a, int b) {
        return a + b;
    }

    public static String convertirAMayusculas(String texto) {
        return texto.toUpperCase();
    }
}

public class Main {
    public static void main(String[] args) {
        int resultadoSuma = Utilerias.suma(5, 3);
        String textoMayusculas =
            Utilerias.convertirAMayusculas("hola");

        System.out.println("Resultado de la suma: " + resultadoSuma);
        System.out.println("Texto en mayúsculas: " + textoMayusculas);
    }
}
```

La clase Utilerias del ejemplo anterior se diseñaría en un diagrama UML así:



Clases Abstractas

¿Qué es una clase abstracta?

Las **clases abstractas**, como su nombre lo indica, son algo abstracto, no representan algo específico. Una clase abstracta se utiliza para modelar conceptos abstractos o genéricos que no tienen una implementación concreta, pero son la base común de otros objetos.

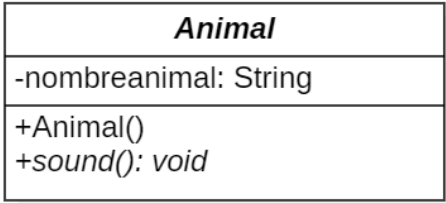
Estas clases no pueden ser instanciadas, por lo que no se pueden crear objetos con ellas y solamente se usan para crear otras clases.

Puede imaginarse a una clase **Animal**, con métodos como sonido, caminar y comer, como una clase base que puede heredar para construir otras clases como un **León** o un **Pájaro**. Ambas van a heredar de la clase **Animal** y cada uno tendrá sus propios métodos, para crear los objetos Leon y Pajaro. De esta manera puede reducirse código duplicado y mejorar la calidad del código.

Por tanto, en la POO, las **clases abstractas** se usan como clases bases de una jerarquía y representan la funcionalidad común de un conjunto diverso de tipos de objetos.

En Java, una clase abstracta y/o sus miembros se define con la palabra reservada **abstract**.

Observe el siguiente ejemplo de una clase Abstracta, con su equivalente en simbología de un diagrama UML:

Codigo Java	Diagrama UML
<pre>public abstract class Animal { private String nombreakimal; public Animal(String nombre){ nombreakimal = nombre; } //metodos abstractos public abstract void sound(); }</pre>	<pre>classDiagram class Animal { -nombreakimal: String +Animal() +sound(): void } class Animal { <i>Animal</i> <i>-nombreakimal: String</i> <i>+Animal()</i> <i>+sound(): void</i> }</pre> Una clase y/o miembro abstracto se redacta en <i>letra cursiva</i>.

Metodo abstracto

Una clase Abstracta puede contener métodos abstractos, los cuales no deben llevar definición/cuerpo, solo definir a su encabezado y finalizarlo con punto y coma (;), ya que su implementación (cuerpo del método) ha de realizarse en cada subclase que herede de la clase abstracta. Además, toda la clase también debe ser marcada como abstracta.

Implementación en Java de un tipo de dato abstracto

Las clases en Java facilitan la creación de ADTs (tipos de datos abstractos), los cuales ocultan su implementación a los clientes (o usuarios de la clase). Un problema en los lenguajes de programación por procedimientos es que el código cliente a menudo depende de los detalles de implementación de los datos utilizados en el código. Esta dependencia tal vez requiera de reescribir el código cliente, si la implementación

de los datos cambia. Los ADTs eliminan este problema al proporcionar interfaces (conjunto de métodos ofrecidos por las clases) independientes de la implementación a sus clientes.

Es posible para el creador de una clase cambiar la implementación interna de esa clase sin afectar a los clientes de esa clase).

III. PROCEDIMIENTO

PARTE 1: Implementacion de Herencia

Descargue los archivos incluidos en los recursos complementarios de este practica 4.

Abra el archivo **POOGuia4_UMLparte1.png**, el cual describe un diagrama UML con los paquetes, clases y sus miembros a desarrollar en este procedimiento.

1. Cargue la aplicación IntelliJ-IDEA y cree un proyecto Java con el nombre **POOGuia4**.
2. Tenga visible el archivo con el diagrama UML de Clases, ya que cada miembro indicado ahí se ira implementado en su proyecto Java.
3. Para iniciar, dentro de la carpeta **src** del proyecto, cree a 2 paquetes, llamado **herencia** y **aplicacion**.
4. Dentro del paquete **herencia**, cree a cada una de las clases indicadas en el UML, pero no defina aun a sus miembros.
5. Para crear el tipo de dato enumerado **Propulsion** indicado en el UML, haga clic secundario sobre el paquete **herencia** y seleccione la opción *New->File*. Asigne al archivo el nombre **Propulsion.java**.

Luego, defina ahí al siguiente código:

```
package herencia;

public enum Propulsion{
    MOTOR,
    VELAS,
    REMOS
}
```

6. Ahora procederá a definir cada relación de herencia indicada en el UML.
7. Para iniciar, modifique el encabezado de la clase Barco, insertando la palabra **extends** y luego, el nombre de su clase base de la cual hereda, que en este caso sera Vehiculo:

```
public class Barco extends Vehiculo
```

8. Abra la definición de la clase **Terrestre**. Ya que esta clase y la clase **Barco** heredan de la misma superclase **Vehiculo**, haga una modificación identifica a la descrita en el paso anterior.
9. Abra la definición de la clase **Moto** y modifique su encabezado para que sea una subclase de la superclase **Terrestre**.
10. Abra la definición de la clase **Camion** y sobre su encabezado, repita el cambio realizado en el paso anterior, porque esta clase hereda también de la clase **Terrestre**.
11. Ahora definirá a los miembros de cada una de las clases del esquema UML.
12. Abra la definición de la clase **Vehiculo** y declare a los campos indicados en el UML para esta clase.

Por cada campo, asigne la visibilidad/estereotipo de acceso indicado en el diagrama de clases.

Ademas, no asigne los valores iniciales, ya que esta inicialización debe realizarse en el metodo constructor correspondiente.

13. Defina el siguiente metodo constructor para la clase **Vehiculo**:

```
public Vehiculo(){
    tipovehiculo = "ninguno";
    nombre = "desconocido";
    marca = "pendiente";
}
```

14. Defina al metodo publico llamado **VerDatos** indicado en el UML y digite la siguiente linea como definicion:

```
System.out.println("Este es un vehiculo genérico, sin nombre ni marca");
```

15. Encapsule al campo **modelo**, para que sea accesible como “solo lectura” al instanciar objetos, digitando el siguiente metodo:

```
public String getTipovehiculo(){
    return tipovehiculo;
}
```

16. Tal como muestra el UML, el campo **tipovehiculo** sera de “solo lectura” al instanciar objetos.

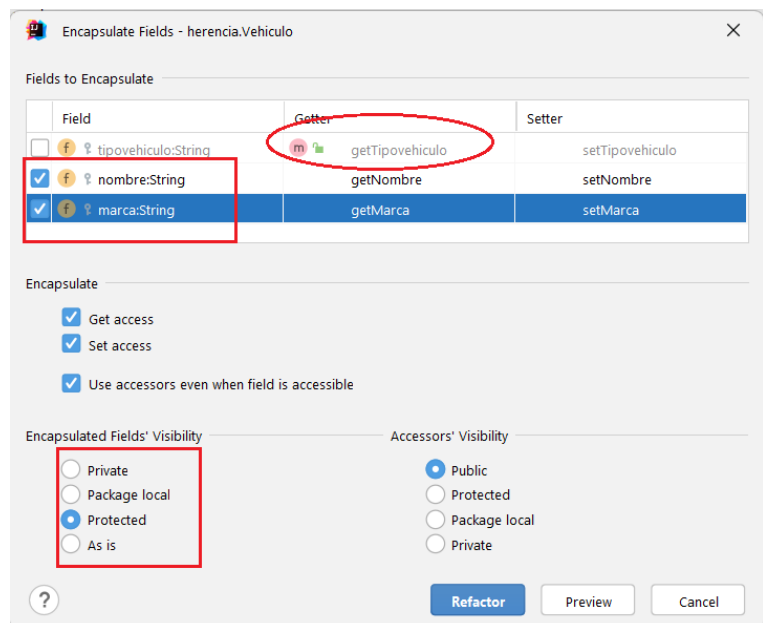
En cambio, los campos restantes (**nombre** y **marca**) si podrán ser leídos y modificados de manera publica al instanciar objetos, por lo que se deberá crear los setter y getter correspondientes.

Ejecute los siguientes pasos para encapsular el campo **marca**:

a. Haga clic secundario sobre el campo **nombre** y del menú contextual, seleccione *Refactor -> Encapsulate Fields*

b. En la ventana del asistente, no marque al campo **tipovehiculo**, porque usted ya lo encapsulo manualmente (con solo el metodo getter, como muestra la foto).

c. En cambio, si debe marcar al campo **marca**. En la lista Encapsulated Fields ‘Visibility’ asegúrese que se creara los getter/setter (Get Access, Set access) y que la visibilidad de los accesores sea publica.



d. Haga clic en botón *Refactor*.

e. Al regresar a la clase, confirme que se crearon los accesores con visibilidad publica para este campo **marca**. Además, en el constructor, verifique que se usa el accesor setter para modificar a campo **marca**.

17. Abra la definición de la clase **Barco**. Declare a su único campo indicado en el UML y recuerde no inicializarlo aún, porque esta acción se realizará en el constructor.
18. Encapsule al campo declarado en el paso anterior, manteniendo su estereotipo *protected*, que sus accesores sean públicos y que tenga ambos métodos (setter, getter).
19. Defina el constructor de la clase **Barco** y asegúrese que tenga la misma *firma de parámetros* indicada en el UML. Luego redacte el siguiente código dentro del metodo constructor de esta clase **Barco**:

```
public Barco(String nom, String marca, Propulsion tipopropulsion){  
    //inicializa campos  
    tipovehiculo = "Barco";  
    propulsion = Propulsion.MOTOR;  
    //modifica campos segun valores de parametros  
    setNombre(nom);  
    setMarca(marca);  
    setPropulsion(tipopropulsion);  
}
```

20. Ahora redacte el metodo llamado **verdatos** con la siguiente definición:

```
public void verdatos(){  
    //ej. Barco Portaaviones Ronald Rigan, Clase Nimitz, Propulsion: Motores  
    String descrip = String.format(  
        "%s %s, %s, Propulsion: %s\n",  
        getTipovehiculo(),getNombre(),getMarca(),  
        getPropulsion().toString());  
    System.out.println(descrip);  
}
```

Observe que el encabezado de este metodo es idéntico al definido en la superclase **Vehiculo**, por lo que este nuevo metodo en la clase **Barco** reemplazara al heredado de la clase base **Vehiculo**.

21. Para comprobar el funcionamiento de la clase derivada **Barco**, haga clic secundario sobre el paquete **aplicacion** y cree a una clase java llamada **Principal**.
22. Al inicio del archivo de la clase **Principal**, importe a todas las clases del paquete **herencia**, digitando la linea:

```
import herencia.*;
```

23. En el cuerpo de la clase, defina al metodo publico estático de inicio **main** y a otro metodo llamado **demoherencia**.
24. Dentro del metodo **demoherencia**, redacte el siguiente código preliminar, el cual comprueba el uso de la clase derivada **Barco**:

```
Barco velero, ronald; //declara objetos de clase derivada Barco  
System.out.println("Demostracion del principio de herencia de clases\n");  
//crea a objetos  
velero = new Barco("Paraiso","Marca Leonard",Propulsion.VELAS);  
velero.verdatos();  
  
ronald = new Barco("Titanic","Clase Olympic",Propulsion.MOTOR);
```

```
//Luego se modifican 2 de los 3 campos, de manera individual
ronald.setNombre("Portaaviones Ronald Reagan");
ronald.setMarca("Clase Nimitz");

ronald.verdatos();
```

25. Ubique al método **main** e invoque en su interior al método **demoherencia**.

Luego, ejecute su aplicación. El resultado será el siguiente:

```
Principal x
"C:\Program Files\Microsoft\jdk-11.0.12.7-hotspot\bin\java.exe" "
Demostracion del principio de herencia de clases

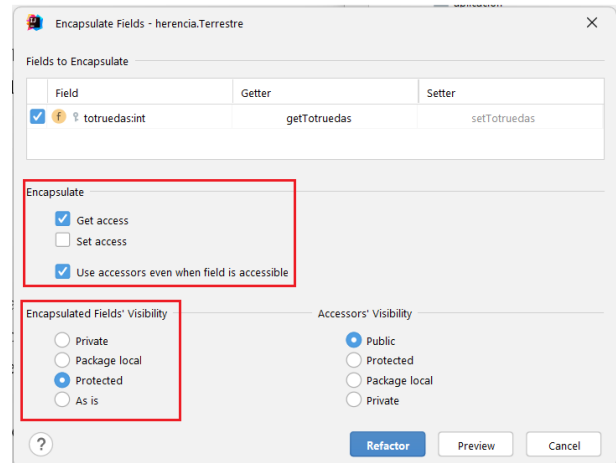
Barco Velero Paraiso, Marca Leonard, Propulsion: VELAS

Barco Portaaviones Ronald Reagan, Clase Nimitz, Propulsion: MOTOR

Process finished with exit code 0
```

26. Para continuar con el esquema de herencia, ahora abra la definición de la clase **Terrestre**.

Declare el único campo indicado en el UML para esta clase. Luego, proceda a encapsularlo, teniendo en cuenta que debe mantener su visibilidad protegida (**protected**) y que solo tenga accesor get (access method).



27. Cambiase a la definición de la clase **Moto**.

28. Inicie los cambios declarando a su único campo indicado en el UML y luego, proceda a encapsularlo (getter/setter).

29. Encapsule al campo heredado **Terrestre.totruedas** con el siguiente setter:

```
public void setTotruedas(int totruedas){
    switch (totruedas){
        case 2: case 3: case 4:
            super.totruedas = totruedas;
            break;
    }
}
```

Importante: Observe el uso de la palabra **super**, que es utilizada para acceder a miembros heredados de una superclase.

30. Elabore el método constructor de esta clase **Moto**, con la firma de parámetros indicada en el UML y con la siguiente definición:

```
public Moto(
    String nom, String marca, int totruedas, double cilindraje){
    super.tipovehiculo = "Moto";
```

```

    totruedas = 2; //por defecto, moto de 2 ruedas
    setNombre(nom);
    setMarca(marca);
    setTotruedas(totruedas);
    setCilindraje(cilindraje);
}

```

31. Finalmente, sobrescriba el metodo **verdatos** (heredado desde la superclase **Vehiculo**) con la siguiente definición:

```

public void verdatos(){
    //ej: Moto Honda ATV, Modelo 2024, 4 ruedas, Cilindrada: 420 CC.
    String descrip = String.format(
        "%s %s, %s, %d ruedas, Cilindrada: %.1f CC.",
        super.getTipovehiculo(), getNombre(), getMarca(),
        getTotruedas(), getCilindraje()
    );
    System.out.println(descrip);
}

```

32. Retorne a la clase **Principal** y dentro del metodo **demoherencia**, inserte al final a las siguientes líneas para demostrar el uso de la subclase **Moto**:

```

Moto honda = new Moto("Honda ATV", "Modelo 2024", 4, 420);
honda.verdatos();

```

33. Ejecute nuevamente la aplicación y en la ultima línea vera el resultado de invocar al metodo verdatos de un objeto de la clase Moto:

```

Moto Honda ATV, Modelo 2024, 4 ruedas, Cilindrada: 420,0 CC.

Process finished with exit code 0

```

34. Guarde los cambios realizados hasta ahora en su proyecto Java.

PARTE 2: Implementacion de Clases Estaticas

35. Abra el archivo **POOGuia4_UMLparte2.png**, con el diagrama UML de las clases a implementar en esta Parte del procedimiento.

Estudie el UML proporcionado. La demostración de uso de las *clases estáticas* consistirá en elaborar una clase estática que contenga los métodos necesarios para operar matemáticamente (sumar, restar, etc.) a números complejos ($a+bi$).

36. En la carpeta **src**, cree el paquete **clasesestaticas** y en su interior, cree a las clases **NumComplejo** y **MateComplejos**, solicitadas en el UML.

37. La clase **NumComplejo** permite abstraer la representación de un numero complejo, que se compone de un número real (r) y otro de tipo complejo (i).

Proceda a declarar a los campos para esta clase indicados en el UML proporcionado, así como también, defina el metodo *asignarvalor*, el cual debe asignar cada parámetro recibido al campo correspondiente de esta clase **NumComplejo**.

Y no olvide que el estereotipo en estos primeros 3 miembros debe ser **private**.

38. Redacte las siguientes definiciones para los constructores de esta clase:

```
public NumComplejo(){
    asignarvalor(0,0);
}

public NumComplejo(double r, double i){
    asignarvalor(0,0);
    this.real = r;
    this.ima = i;
}
```

39. Con la ayuda del editor del IDE, encapsule el par de campos de esta clase **NumComplejo**.

Verifique que, en los métodos existentes de la clase, ya usan a los setter definidos en este paso.

40. Proceda a crear al metodo **vervalor** según el prototipo dado en el UML. Luego, digite el siguiente código para su definición:

```
//String con representac. de num. complejo almacenado en objeto
String res="";
//parte real del num. complejo
res = String.format("%.2f",real);

//parte imaginaria del num. complejo
//si parte imaginaria es positivo
if (ima >= 0) res += "+";

res += String.format("%.2fi", ima);

return res;
```

41. Para demostrar el funcionamiento de la clase **NumComplejo**, retorne a la clase Principal. Importe a todas las clases del paquete **clasesestaticas**.

42. Cree un metodo estático con el nombre **democlaseestatica** y redacte el siguiente código como definición para el mismo:

```
//declara 4 instancias de clase NumComplejo
NumComplejo a, b, c, d;
//define valor a cada instancia
a = new NumComplejo(); // 0+0i
b = new NumComplejo(3, 8); // 3+8i
c = new NumComplejo(-2, -5); // -2-5i
```

```
d = new NumComplejo(0,-4); // 0 - 4i
//imprime valor almacenado de cada num complejo
System.out.printf("a = %s \nb = %s \nc = %s \nd = %s",
    a.vervalor(),b.vervalor(),c.vervalor(),d.vervalor());
```

43. Ubique al metodo **main**. Comente a la llamada del ultimo metodo usado en las demostraciones anteriores y luego digite la llamada a su nuevo metodo **democlaseestatica**. El resultado sera el siguiente:

```
Principal x
"C:\Program Files\Microsoft\jdk-11.0.12.7-hotspot\bin\java.exe"
a = 0,00+0,00i
b = 3,00+8,00i
c = -2,00-5,00i
d = 0,00-4,00i
Process finished with exit code 0
```

44. Retorne a la clase **NumComplejo**. Ahí implemente al metodo **getConjugada**. Luego, digite la siguiente definición para este ultimo metodo del UML para esta clase:

```
// por ej, la Conjugada del Complejo -4+3i es -4-3i
NumComplejo temp = new NumComplejo();
//se genera un numero complejo, resultado de invertir signo
//de la parte imaginaria del valor almacenado en objeto
temp.setReal(getReal());
temp.setIma(-1 * getIma());
return (temp); //retorna conjugada de num. complejo actual
```

45. Retorne a la definición de la clase **Principal**. Ubique al metodo **democlaseestatica** e incluya las siguientes líneas a su contenido actual:

```
e=b.getConjugada(); //devuelve objeto con conjugada de objeto b
System.out.printf("\nla conjugada de complejo b es %s\n",
    e.vervalor());
```

46. Ejecute nuevamente a su aplicación y verifique que se le retorna que el numero complejo b tiene una conjugada de 3,00-8,00i
47. Con el paso anterior, usted ya tiene la capacidad de representar números complejos y por lo tanto, procederá a crear una clase estática, la cual sera una *calculadora para operar números complejos*. Esta nueva clase definirá métodos estáticos y el uso de objetos de la clase NumComplejo para ejecutar sus operaciones y resultados.
48. Ingrese a la definición de la clase **MateComplejos**. Ya que la clase **NumComplejo** pertenece al mismo paquete, no sera necesario importarla.

49. Proceda a digitar al encabezado del metodo **Suma**. Luego, justo después de la palabra **public**, inserte la palabra **static**. Este metodo sera estático, lo que significa que para invocarlo, no se deberá crear instancias de esta clase, solo deberá incluirse previamente al nombre de su clase.

50. Digite el siguiente cuerpo de instrucciones a este metodo estático llamado **Suma**:

```
NumComplejo res = new NumComplejo();
res.setReal(n1.getReal() + n2.getReal());
res.setIma(n1.getIma() + n2.getIma());
return res;
```

51. Retorne al metodo **democlasescomplejas** y al final de su código actual, inserte las siguientes líneas:

```
//objetos para recibir resultados de metodos estaticos
NumComplejo suma, resta, prod, div, pot;

suma = MateComplejos.Suma(b, c);
System.out.printf("La suma de %s con %s es %s\n",
    b.vervalor(),c.vervalor(),suma.vervalor());
```

52. Ejecute nuevamente su aplicación y verifique que la ultima linea del resultado devuelve:

La suma de 3,00+8,00i con -2,00-5,00i es 1,00+3,00i

Gracias al metodo estático Suma, este devuelve un nuevo objeto, con el resultado de la suma de ambos números complejos.

53. Vaya nuevamente a la clase **MateComplejos**. Tome de modelo a la definición del metodo **Suma** para que usted mismo defina por completo al metodo estatico **Resta** indicado en el UML.

54. Retorne al metodo **democlasescomplejas** y digite al final al siguiente conjunto de instrucciones:

```
resta = MateComplejos.Resta(b,d);
System.out.printf("La resta de %s con %s es %s\n",
    b.vervalor(),d.vervalor(),resta.vervalor());
```

55. Vuelva a ejecutar su aplicación y verifique la resta de los números complejos **b** y **d** sea la correcta.

56. Vuelva a la clase **MateComplejos** y con su conocimiento de algebra con números complejos, redacte su propia definición para el metodo estático Multiplicar indicado en el UML.

57. Modifique el metodo democlasescomplejas, para incluir una demostración del metodo multiplicar, utilizando al par de números complejos: 4-6i y -3+2i.

58. Guarde los cambios realizados en su proyecto.

PARTE 3: Clases Abstractas

59. Para implementar un ejemplo del concepto de abstracción, se presenta el siguiente caso de análisis:

Caso de Estudio:

Los medios de almacenamiento de una computadora son dispositivos de hardware que permiten guardar, recuperar y gestionar información digital, tanto de forma temporal como permanente.

Se proporciona a continuación un conjunto reducido de los parámetros de funcionamiento de 2 de los dispositivos de almacenamiento de datos, denominados Hybrid Hard Drive (HDD) y Solid State Drive (SSD):

Hybrid Hard Drive (HDD):

Capacidad Almacenamiento	Rendimiento	Revoluciones por minuto (RPM)	Platos
1, 2 ... 10 TB	150 - 180 MB/s	5400, 7200 o 10000	1 - 5

Solid State Drive (SSD):

Interfaz	Capacidad Almacenamiento	Rendimiento
SATA	256 o 512 GB	190 - 600 MB/s
PCI express	1 - 4 TB	3500 - 7000 MB/s

60. De acuerdo con la clasificación de datos del *Caso de Estudio* anterior, se necesita implementar un sistema en Java que permita registrar la información de los dispositivos electrónicos HDD y DSS.

61. Para iniciar esta parte del procedimiento, abra el archivo **POOGuia4_UMLparte3.png**

62. Dentro de la carpeta `src`, cree otro paquete bajo el nombre `clasesabstractas`.

63. Luego, de acuerdo con el UML, cree a las clases indicadas dentro de este paquete.

64. Ingrese a la definición de la clase `UnidAlmac`. Observe que en el UML, el nombre de esta clase esta escrito en cursiva, lo que significa que usted de incluir la palabra **abstract** antes de la palabra `class`, así:

```
public abstract class UnidAlmac
```

65. Continúe con la declaración de los campos. Recuerde que los miembros marcados con el estereotipo (#) son protegidos (incluir la palabra `protected` en su declaración).

66. Según el UML, el nombre del único método de esta clase también esta resaltado en cursiva, lo que significa que debe insertar la palabra clave `abstract` justo luego de la palabra `public` en el encabezado de su declaración, así:

```
public abstract String mostrarparametros(){  
}
```

Justo en ese momento, se generara un error. Colóquese encima del encabezado sombreado en rojo y se informara este error: *no abstract method cannot have a body*

Esto significa que al marcar como abstracto a este metodo **mostrarparametros**, este ya no puede tener una definición en la clase abstracta, por lo que usted debe reemplazar la pareja de llaves { } correspondiente por un (;), definiendo así a un *prototipo de metodo*, el cual deberá ser implementado obligatoriamente por cualquier subclase que herede de esta clase abstracta **UnidAlmac**.

67. Localice la definición de la clase **HDD** y declare a sus campos indicados en el UML.

Cuide de aplicar a cada campo el esteotipo/visibilidad **private**. Y, además, no aplique aun la inicialización de sus valores indicados en el UML, porque este lo hará en un paso posterior del procedimiento.

68. Ahora convertirá a esta clase HDD como subclase de la clase abstracta **UnidAlmac**. Para ello, en el encabezado de la clase HDD, luego del nombre de esta clase, inserte la expresión **extends UnidAlmac**.

```
public class HDD extends UnidAlmac
```

Justo en ese momento se marcará un error. Coloque el ratón sobre el encabezado de la clase. Vera que se solicita redactar la definición para el metodo abstracto **mostrarparametros**, que fue heredado de la superclase **UnidAlmac**.

69. Para cumplir la solicitud del paso anterior, retorne a la superclase **UnidAlmac** y haga una copia del prototipo del metodo **mostrarparametros**.

Luego, retorne a la subclase **HDD** y pegue el prototipo justo después de los campos ya declarados. Ahora, borre la palabra **abstract** y reemplace el (;) final por una pareja de llaves { }.

Con esto, usted ha implementado el metodo abstracto heredado de la superclase.

70. El error sera cambiado por otro, debido a que este metodo **mostrarparametros** debe retornar un *String*.

Para solucionarlo temporalmente, digite la siguiente línea de código como cuerpo de este metodo:

```
return null;
```

Mas adelante definirá el cuerpo real de este metodo.

71. Antes de la definición del metodo **mostrarparametros**, digite a los siguientes métodos (getter y setter) para acceder al campo protegido heredado de la superclase **UnidAlmac**:

```
public int getCapac(){
    //devuelve valor de campo (en GigaBytes)
    return super.capac;
}

public void setCapac(int capac){
    //verifica si capacidad recibida es apropiada
    // En GB: 256, 512, 1024 (1 TB), 2048 (2 TB) o 4096 (4 TB)
    switch (capac){
        case 256: case 512:
        case 1024: case 2048: case 4096:
            super.capac = capac; //modifica campo
            break;
    }
}
```

```
}  
}
```

Notas:

- Observe el uso de la palabra **super**, con el cual se referencia desde una subclase a miembros proporcionados desde la superclase.
- Confirme que se implementan las reglas de rangos de valores descritos en el Caso de Estudio presentado al inicio de esta parte del procedimiento para este tipo de dispositivo (HDD), los cuales son distintos para los dispositivos (SDD).

72. Justo antes del metodo **mostrarparametros**, inserte la siguiente definición para el metodo **getCapacidadHDD**:

```
public String getCapacidadHDD(){  
    //retorna valor en GB (para 256 o 512 GB) o  
    //en TB (para resto de valores aceptados)  
    int totaltb; //total de TeraBytes  
    String valorcapac = "";  
    switch (capac){  
        case 256: case 512:  
            valorcapac = String.valueOf(capac) + " GB";  
            break;  
        default:  
            totaltb = capac/1024;  
            valorcapac = String.valueOf(totaltb)+" TB";  
        }  
    return valorcapac;  
}
```

73. Justo después de la declaración de campos, cree al metodo **inicializarhdd** indicado en el UML y digite ahí a las siguientes instrucciones:

```
// En GB: 256, 512, 1024(1 TB), 2048(2 TB) o 4096(4 TB)  
capac = 256;  
// rango: 150 - 180 MB/s  
rendim = 150; //150 MB/s  
// rango: solamente 5400, 7200 (predeterminado), 10000  
rpm = 7200;  
// rango: de 1 a 5. predeterminado 4  
platos = 4;
```

74. Justo después del metodo **inicializarhdd**, ahora redacte a las siguientes definiciones de los métodos constructores para esta clase **HDD**:

```
public HDD(){  
    inicializarhdd();  
}
```

```
public HDD(int capac,int rendim,int rpm,int platos){
    inicializarhdd();
    //intenta actualizar campos
    setCapac(capac);
    super.rendim = rendim;
    this.rpm = rpm;
    this.platos = platos;
}
```

75. Ubique al metodo **mostrarparametros**, borre la linea que retorna null y a cambio, digite la siguiente definición para este metodo:

```
//ej. de string a retornar:
// HDD: Capacidad(512 GB), Rendimiento(160 MB/s), 5400 rpm, 3 Platos
// HDD: Capacidad(2 GB), Rendimiento(175 MB/s), 10000 rpm, 4 Platos
String resul;
resul = String.format(
    "Capacidad (%s), Rendimiento(%d MB/s), %d rpm, %d Platos",
    getCapacidadHDD(),rendim,rpm,platos);
return resul;
```

76. Para comprobar la funcionalidad actual de la clase HDD, retorne a la clase **Principal**.

77. Importe a las clases del paquete clasesabstractas, con la sentencia:

```
import clasesabstractas.*;
```

Además, importe a la clase `java.util.Scanner`

78. Cree un nuevo metodo estático, con el nombre **demoabstraccion1**. Redacte ahí a la siguiente definición:

```
//comprobara implementacion de herencia con abstracion de clase
Scanner teclado = new Scanner(System.in);
HDD Kington, Seagate; //objetos de clase derivada HDD
Kington = new HDD(); //crea instancia con valores predeterminados
//crea objeto, asignando valores especificos
Seagate = new HDD(2048,175,5400,3);
//Imprime los valores almacenados en cada objeto de clase HDD
System.out.println("Estado inicial de par de discos duros (HDD):");
System.out.println("* Kington: "+Kington.mostrarparametros());
System.out.println("* Seagate: "+Seagate.mostrarparametros());
```

79. Ubique la definición del metodo **main**. Convierta en comentario a la llamada del último metodo usado en pruebas anteriores e invoque al metodo **demoabstraccion1**.

80. Ejecute la aplicación.

Principal x

Estado inicial de par de discos duros (HDD):

* Kington: Capacidad (256 GB), Rendimiento(150 MB/s), 7200 rpm, 4 Platos
* Seagate: Capacidad (2 TB), Rendimiento(175 MB/s), 5400 rpm, 3 Platos

Process finished with exit code 0

81. Observe que, gracias al metodo setter **setCapac**, solo se verifica el valor que intenta asignar como capacidad de memoria (en GigaBytes).
82. Para demostrarlo, dentro del metodo **demoabstraccion1** localice la linea donde se crea al objeto **Seagate** y reemplace la secuencia de argumentos (2048, 175, 5400, 3) utilizada por una secuencia incoherente de valores como (100, -6, 0, 43).
83. Vuelva a ejecutar su aplicación y obtendrá el siguiente resultado:

Principal x

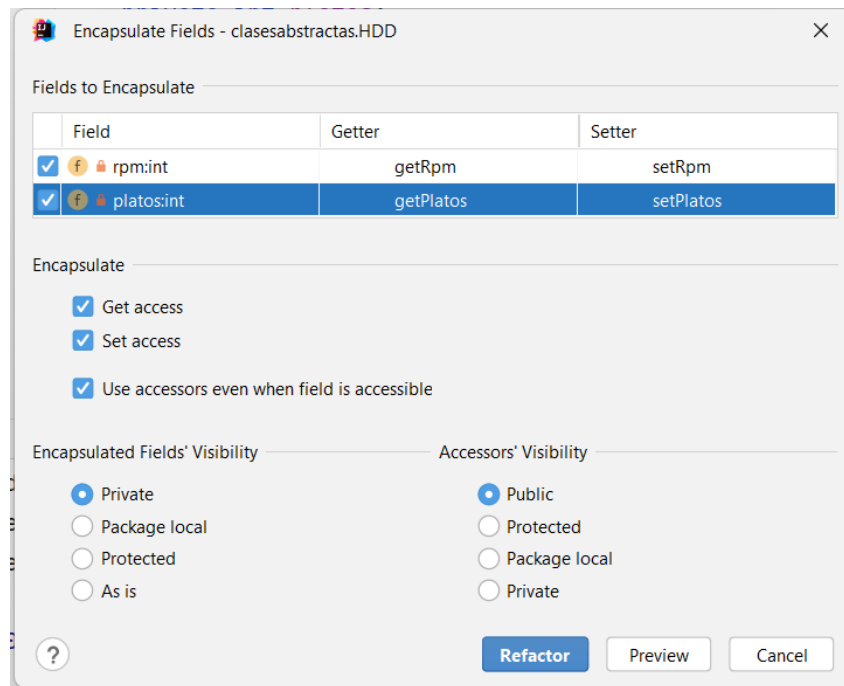
Estado inicial de par de discos duros (HDD):

* Kington: Capacidad (256 GB), Rendimiento(150 MB/s), 7200 rpm, 4 Platos
* Seagate: Capacidad (256 GB), Rendimiento(-6 MB/s), 0 rpm, -43 Platos

84. Ahora completaremos la lógica de valores aceptados para los campos restantes de la clase HDD.
85. Para el valor del *rendimiento*, tome en cuenta que, según las tablas proporcionadas en el Caso de Estudio de esta parte del procedimiento, los rangos de rendimiento entre los HDD y SSD son diferentes. Así que el campo *rendim* de la clase **UnidAlmac** no puede encapsularse en esta superclase, sino que debe hacerse en cada clase derivada, aplicando así a una “lógica/gestion de negocios” distinta.
86. Retorne a la definición de la clase **HDD**. Ubique el cursor antes de la definición del metodo *mostrarparametros*, luego inserte a los siguientes métodos *getter* y *setter* para el campo protegido **rendim**:

```
public int getRendim(){  
    return rendim;  
}  
  
public void setRendim(int rendim){  
    //acepta valor solo entre (150 a 180 MB/s)  
    if(rendim>=150 && rendim<=180)  
        super.rendim = rendim;  
}
```

87. Para los campos locales (*rpm* y *platos*) de la clase HDD, utilice el editor del IDE para encapsular a este par de datos:



88. Para implementar la lógica de negocios para los valores aceptados por el campo **rpm**, localice el metodo setter generado para este campo y modifíquelo con la siguiente definición:

```
public void setRpm(int rpm) {
    // rango: solamente 5400, 7200 (predeterminado), 10000
    switch (rpm){
        case 5400: case 7200: case 10000:
            this.rpm = rpm;
            break;
    }
}
```

89. De manera similar al paso anterior, localice al setter generado para el campo **platos** y modifíquelo con la definición a continuación:

```
public void setPlatos(int platos) {
    if(platos>=1 && platos<=5)
        this.platos = platos;
}
```

90. Por criterio de ordenamiento de código, mueva la definición del metodo **mostrarparametros** como ultimo metodo de la clase **HDD**.

91. Ubique la definición de la sobrecarga del constructor de la clase **HDD** y cambie la asignación directa del campo `super.rendim` por la llamada al metodo *setter* para este campo, de la siguiente manera:

```
setRendim(rendim);
```

92. Retorne a la clase Principal y vuelva a ejecutar la aplicación. Verifique que todos los campos (heredados y locales) de la clase HDD no aceptan los argumentos enviados, haciendo que el objeto **Seagate** conserve los valores predeterminados definidos en su metodo constructor:

```
Principal x
Estado inicial de par de discos duros (HDD):
* Kington: Capacidad (256 GB), Rendimiento(150 MB/s), 7200 rpm, 4 Platos
* Seagate: Capacidad (256 GB), Rendimiento(150 MB/s), 7200 rpm, 4 Platos
```

93. Restaure la secuencia de argumentos enviados al crear el objeto **SeaGate**:

```
Seagate = new HDD(2048,175,5400,3);
```

94. Complete la definición del metodo **demoabstraccion1** con las siguientes líneas:

```
//intenta modificar algunos campos con valores dados por usuario
System.out.print("\nIngrese la nueva capacidad (en GB) de Kington ? ");
Kington.setCapac(teclado.nextInt());
System.out.println("Nuevo estado del HDD Kington:");
System.out.println("* Kington: "+Kington.mostrarparametros());
System.out.print(
    "\nAhora ingrese el nuevo rendimiento (en MB/s) de Seagate ? ");
Seagate.setRendim(teclado.nextInt());
System.out.println("Nuevo estado del HDD Seagate:");
System.out.println("* Seagate: "+Seagate.mostrarparametros());
```

95. Vuelva a ejecutar la aplicación e ingrese los valores para alterar los datos almacenados en ambos objetos.
96. Haga varias pruebas de ejecución, ingresando datos apropiados y hasta incoherentes en los campos y así verificar si los **setter** de los campos funcionan correctamente.

97. Ahora cámbiese a la definición de la clase **SSD**. Cree ahí al campo local indicado en el UML, así:

```
private char interfaz; //'s': SATA o 'p': PCI express
```

98. Haga los cambios necesarios para que esta clase **SSD** se derive de la clase **UnidAlmac**.

Cuando haga el cambio anterior, no olvide redactar una definición temporal para el metodo heredado **mostrarparametros** y en la cual se retorne el valor **null**.

99. Con ayuda del editor del IDE, encapsule al campo local **interfaz**.

Luego, ubique al metodo **setter** creado para este campo y reemplace su contenido con el siguiente código:

```
public void setInterfaz(char interfaz) {
    //convierte parametro en minuscula
    interfaz = Character.toLowerCase(interfaz);
    //si nuevo tipo interfaz es distinto al actual
    if(interfaz!=this.interfaz){
        //inicializa resto de campos segun tipo interfaz
        switch (interfaz){
            case 's': // 's': SSD SATA
                //para SATA, estos son los valores iniciales:
```



```

        this.interfaz = interfaz; //actualiza campo interfaz
        capac = 256; //(256, 512) GB
        rendim = 190; // (190 - 600) MB/s
        break;
    case 'p': // 'p': PCIe (pci express)
        //para PCIe, estos son los valores iniciales:
        this.interfaz = interfaz; //actualiza campo interfaz
        capac = 1; //(1 - 4) TB
        rendim = 3500; // (3500 - 7000) MB/s
        break;
    } //fin tipointerfaz
}
}

```

100. Para cada campo heredado de la superclase, proceda a digitar los siguientes *getter* y *setter*, los cuales implementan la lógica de negocios para los dispositivos SSD incluidos en el *Caso de Estudio* de esta Parte del procedimiento:

```

public int getCapac(){
    return capac;
}

public void setCapac(int capac){
    //verifica que capacidad este en rango segun tipo interfaz
    switch (getInterfaz()){
        case 's': // SATA
            if(capac==256 || capac==512) //GB
                super.capac = capac;
            break;
        case 'p': // PCIe
            if(capac>=1 && capac<=4) //TB
                super.capac = capac;
            break;
    }
}

public int getRendim(){
    return super.rendim;
}

public void setRendim(int rendim){
    //verifica que rendimiento este en rango segun tipo interfaz
    switch (getInterfaz()){
        case 's': // SATA
            if(rendim>=190 && rendim<=600)
                super.rendim = rendim;
            break;
        case 'p': // PCIe
            if(rendim>=3500 && rendim<=7000)
                super.rendim = rendim;
            break;
    }
}
}

```

101. Justo luego de la declaración del campo **interfaz**, inserte a los siguientes métodos constructores para esta clase **SSD**:

```
public SSD(){ //constructor
    this.interfaz = '-';
    setInterfaz('s'); //define valores para interfaz SATA
}

//sobrecarga de constructor
public SSD(char interfaz, int capac, int rendim){
    //define valores para interfaz SATA
    setInterfaz('s');

    //actualiza campos con parametros recibidos
    setInterfaz(interfaz);
    setCapac(capac);
    setRendim(rendim);
}
```

102. Ubique al metodo **mostrarparametros** y reemplace su contenido a ejecutar por las siguientes líneas:

```
//ej. de string a retornar:
// DSS: interface SATA, Capacidad 256 GB, Rendimiento(300 MB/s)
// DSS: interface pci express, Capacidad 3 GB, Rendimiento(5000 MB/s)
String resul = "";
String nominterfaz;
switch (interfaz){
    case 's':
        nominterfaz = "SATA";
        resul = String.format(
            "interface %s, Capacidad %d GB, Rendimiento(%d MB/s)",
            nominterfaz, getCapac(), getRendim());
        break;
    case 'p':
        nominterfaz = "PCIe";
        resul = String.format(
            "interface %s, Capacidad %d TB, Rendimiento(%d MB/s)",
            nominterfaz, getCapac(), getRendim());
        break;
}
return resul;
```

103. Ya solamente falta demostrar el funcionamiento de la subclase DSS. Para hacerlo, retorne a la clase **Principal**.
104. Cree a un método estático llamado **demoabstraccion2** y redacte ahí al siguiente código:

```
Scanner teclado = new Scanner(System.in);
SSD disco1, disco2;
disco1 = new SSD(); //interfaz SATA predeterminada
disco2 = new SSD('P', 3, 5000); //asigna interfaz PCIe
System.out.println("Estado inicial de par de medios (SSD):");
```

```

System.out.println("* SSD 1: "+disco1.mostrarparametros());
System.out.println("* SSD 2: "+disco2.mostrarparametros());
//intenta modificar algunos campos con valores dados por usuario
System.out.print("\nIngrese la nueva capacidad (256 o 512) GB de SSD 1 ? ");
disco1.setCapac(teclado.nextInt());
System.out.println("Nuevo estado del SSD 1:");
System.out.println("* SSD 1: "+disco1.mostrarparametros());
System.out.print(
    "\nIngrese letra (s: SATA o p: PCIe) para cambiar tipo Interfaz del SSD 2? "
);
disco2.setInterfaz(teclado.next().charAt(0));
System.out.println("Nuevo estado del SSD 2:");
System.out.println("* SSD 2: "+disco2.mostrarparametros());

```

105. Localice al metodo **main**, desactive la llamada al metodo **demoabstraccion1** e incluya la llamada al metodo **demobstraccion2**.

106. Vuelva a ejecutar su aplicación. Observe el *estado* (dado por la combinación de valores actuales de los campos de un objeto) de cada objeto (**disco1** y **disco2**).

Al cambiar valores a campos, asigne valores incoherentes según la lógica de negocios. Los objetos no deberán aceptar estos valores y sus campos no serán modificados.

IV. EJERCICIOS COMPLEMENTARIOS.

EJERCICIO 1 (30%):

Complete el ejemplo desarrollado en la Parte 1 del procedimiento de esta practica, creando una definición de miembros pendientes (Constructor, reemplazo de metodo heredado **verdatos** y definición de los setter/getter apropiados) para la clase derivada **Camion**.

Finalmente, en el metodo **demoherencia**, inserte las líneas necesarias para demostrar la creación de 3 objetos de esta subclase **Camion**: el 1ero con datos apropiados, el objeto 2 con cantidad de ruedas invalida y el ultimo objeto, se crea y luego se le modifican sus campos de manera individual.

Restricciones:

- No se debe modificar la cantidad ni nombres de campos ya indicados en el uml para este clase derivada ni del resto de clases del Esquema de Herencia de Clases usadas en el ejemplo del procedimiento.
- Se debe investigar los criterios para definir el tonelaje permitido y establecer un tonelaje por defecto. Estos criterios deben aplicarse en el accesor setter para el campo correspondiente para esta clase **Camion**.
- La configuración de ruedas para un camion no es la misma que para una moto, por lo que debe investigar los estándares relacionados al conteo permitido de ruedas de este tipo de vehículos, definir un total de ruedas predeterminado y definir las normas en el metodo setter de acceso al campo **totruedas**.

EJERCICIO 2 (35%):

Complete el ejemplo desarrollado en la Parte 2 del procedimiento, creando una definición para los métodos aun no implementados de la clase estática **MateComplejos**.

Restricciones:

- No se debe modificar a ninguno de los miembros de las clases del paquete **clasesestaticas**, ya desarrollados durante el procedimiento.
- No se debe incluir mas miembros de los indicados en el UML brindado para la Parte 2 del procedimiento.

EJERCICIO 3 (35%):

Para el ejemplo de la Parte 3 del procedimiento de esta practica, haga una investigación acerca del diseño y funcionamiento de las memorias microSD. Luego identifique a 2 parametros de funcionamiento que correspondan solamente a este tipo de dispositivos de almacenamiento, es decir que no esten definidos en los campos de la clase abstracta usada en el procedimiento. Y también, identifique el equivalente de medidas cubiertas en los campos existentes de la clase estática.

Finalmente, modificar la aplicación del ejemplo para crear una subclase de la clase abstracta UnidAlmac para almacenar datos de las memorias microSD.

Realice una demostración de su clase para gestión de datos de las microSD, creando 3 objetos distintos, permitiendo a usuario ingresar datos para los mismos y así probar los getter/setter diseñados para implementar la lógica de negocios acerca de estos dispositivos de almacenamiento.

V. REFERENCIA BIBLIOGRAFICA.

Como Programar en Java. (2004). 5th ed. Deitel&Deitel.

Paquetes. (2016). Webtaller.com. Accedido 28 Octubre 2016, de <http://www.webtaller.com/manual-java/paquetes.php>